

Corrigé du TD

Calcul de complexité algorithmique

Cas itératif • Cas récursif

CPGE — filières MP / PSI | Informatique pour tous

Partie I — Le cas itératif : boucles for imbriquées

Exercice 1 — Échauffement — compter avant de majorer

Solution.

- (a) n exécutions. Complexité $\Theta(n)$.
- (b) $\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = n^2$. Complexité $\Theta(n^2)$.
- (c) mn exécutions. Complexité $\Theta(mn)$.
- (d) $\sum_{i=0}^{n-1} \sum_{j=0}^4 \sum_{k=0}^{n-1} 1 = 5n^2$. Complexité $\Theta(n^2)$: la boucle de longueur fixe 5 n'apporte qu'une constante multiplicative, invisible dans le Θ .

Synthèse. Non : mn n'est $\Theta(n^2)$ que si $m = \Theta(n)$. Si m est une constante, la complexité est $\Theta(n)$; si $m = n^2$, elle est $\Theta(n^3)$. Dès qu'un algorithme dépend de *plusieurs* paramètres, la complexité doit s'exprimer en fonction de *tous* ces paramètres : écrire $\Theta(mn)$ et non $\Theta(n^2)$.

Exercice 2 — Boucles triangulaires

Solution. 1. Pour (a) : $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$. Pour (b) : la boucle interne `range(i,n)` fait $n-i$ tours,

d'où $\sum_{i=0}^{n-1} (n-i) = \sum_{j=1}^n j = \frac{n(n+1)}{2}$. La différence vaut $\frac{n(n+1)}{2} - \frac{n(n-1)}{2} = n$: c'est exactement le cas $j = i$, présent dans (b) et absent de (a). Dans les deux cas la complexité est $\Theta(n^2)$, avec la même constante dominante $\frac{1}{2}$.

2. La boucle exécute `s += 1` une fois par triplet (i, j, k) tel que $1 \leq i \leq j \leq k \leq n$. Le nombre de tels triplets est le nombre de combinaisons *avec répétition* de 3 éléments parmi n :

$$\binom{n+3-1}{3} = \binom{n+2}{3} = \frac{n(n+1)(n+2)}{6} \sim \frac{n^3}{6}.$$

Complexité $\Theta(n^3)$.

Vérification directe.
$$\sum_{i=1}^n \sum_{j=i}^n (n-j+1) = \sum_{i=1}^n \sum_{\ell=1}^{n-i+1} \ell = \sum_{i=1}^n \frac{(n-i+1)(n-i+2)}{2} = \sum_{p=1}^n \frac{p(p+1)}{2} = \frac{n(n+1)(n+2)}{6}.$$

3. Pour p boucles imbriquées avec $1 \leq i_1 \leq \dots \leq i_p \leq n$, le nombre d'itérations est $\binom{n+p-1}{p} \sim \frac{n^p}{p!}$, soit $\Theta(n^p)$ à p fixé.

Exercice 3 — Indices strictement croissants

Solution. 1. Dans le pire des cas (aucun triplet de somme nulle), le test est effectué une fois par triplet $0 \leq i < j < k \leq n-1$, soit

$$\binom{n}{3} = \frac{n(n-1)(n-2)}{6} \sim \frac{n^3}{6} \text{ tests, } \quad \text{complexité } \Theta(n^3).$$

2. Dans le meilleur des cas, $T[0]+T[1]+T[2] == 0$ et l'algorithme renvoie après un seul test : complexité $\Theta(1)$. Les complexités du meilleur et du pire cas n'étant pas du même ordre, il n'existe aucune fonction g telle que le coût soit $\Theta(g(n))$ pour toute entrée. On doit donc préciser : « $\Theta(n^3)$ dans le pire des cas », ou se contenter de « $O(n^3)$ ».

3. Pour (b) : $\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} n = n \binom{n}{2} = \frac{n^2(n-1)}{2} \sim \frac{n^3}{2}$, soit $\Theta(n^3)$. L'ordre de grandeur est le même que (a) ; seule la constante change ($\frac{1}{2}$ au lieu de $\frac{1}{6}$). *Conclusion utile* : remplacer `range(j+1, n)` par `range(n)` ne fait pas changer de classe de complexité.

Exercice 4 — Bornes non linéaires et pas variable

Solution. (a) $\sum_{i=0}^{n-1} i^2 = \frac{(n-1)n(2n-1)}{6} \sim \frac{n^3}{3}$. Complexité $\Theta(n^3)$.

(b) La boucle `range(1, n+1, i)` parcourt $1, 1+i, 1+2i, \dots$ et effectue $\lceil n/i \rceil$ tours. Donc

$$N(n) = \sum_{i=1}^n \left\lceil \frac{n}{i} \right\rceil, \quad \sum_{i=1}^n \frac{n}{i} \leq N(n) \leq \sum_{i=1}^n \left(\frac{n}{i} + 1 \right) = nH_n + n.$$

Comme $H_n = \ln n + \gamma + o(1)$, on obtient $N(n) \sim n \ln n$, soit $\Theta(n \log n)$.

(c) La boucle `while` interne s'exécute tant que $j^2 \leq i$; elle fait donc $\#\{j \geq 0 : j^2 \leq i\} = \lfloor \sqrt{i} \rfloor + 1$ tours. D'où

$$N(n) = \sum_{i=1}^n (\lfloor \sqrt{i} \rfloor + 1).$$

Par comparaison série-intégrale, $\int_0^n \sqrt{t} dt \leq \sum_{i=1}^n \sqrt{i} \leq \int_1^{n+1} \sqrt{t} dt$, donc $\sum_{i=1}^n \sqrt{i} \sim \frac{2}{3}n^{3/2}$ et $N(n) \sim \frac{2}{3}n^{3/2}$: complexité $\Theta(n^{3/2})$.

Exercice 5 — Progression géométrique : attention aux fausses évidences

Solution. 1. Avec $n = 2^p$, la variable `i` prend les valeurs $1, 2, 4, \dots, 2^{p-1}$ (elle s'arrête dès que $i \geq n$). Le coût total est

$$\sum_{t=0}^{p-1} 2^t = 2^p - 1 = n - 1.$$

Complexité $\Theta(n)$, et non $\Theta(n \log n)$: la boucle externe fait bien $\log_2 n$ tours, mais les tours ont des coûts très inégaux et la somme géométrique est dominée par son *dernier* terme. Le résultat n'est donc pas surprenant une fois la somme écrite — il l'est seulement si l'on multiplie hâtivement.

2. Ici chaque tour coûte n , indépendamment de i : coût total $= p \cdot n = n \log_2 n$, soit $\Theta(n \log n)$.

3. *Moralité*. Le coût d'une boucle est la **somme** des coûts de ses tours, jamais le produit « nombre de tours $\times$ coût du dernier tour ». Ce produit n'est une bonne approximation que lorsque les tours ont un coût du même ordre (cas (b)) ; il surestime lourdement dans le cas d'une progression géométrique (cas (a)).

4. Avec $i = i * i$ et $i_0 = 2$: après t tours $i = 2^{2^t}$. La boucle s'arrête au premier $t = K$ tel que $2^{2^K} \geq n$, donc elle effectue $\Theta(\log \log n)$ tours. Le coût total vaut $\sum_{t=0}^{K-1} 2^{2^t}$, somme à croissance doublement exponentielle, donc dominée par son dernier terme :

$$2^{2^{K-1}} \leq \sum_{t=0}^{K-1} 2^{2^t} \leq 2 \cdot 2^{2^{K-1}}.$$

Or $2^{2^{K-1}} < n$ et $(2^{2^{K-1}})^2 = 2^{2^K} \geq n$, donc $\sqrt{n} \leq 2^{2^{K-1}} < n$. Le coût total est donc $O(n)$ et $\Omega(\sqrt{n})$, mais il n'existe pas de c tel que ce coût soit $\Theta(n^c)$: il oscille selon la position de n entre deux valeurs consécutives 2^{2^t} .

Partie II — Le cas itératif : boucles while

Exercice 6 — Les trois schémas fondamentaux

Solution. 1.

- (a) après k tours $x = n - k$; arrêt quand $n - k = 0$: exactement n tours, $\Theta(n)$.
- (b) après k tours $x = n - kc$; arrêt au premier k tel que $n - kc \leq 0$, soit $k = \lceil n/c \rceil$ tours, $\Theta(n)$.
- (c) on montre par récurrence que $x = \lfloor n/2^k \rfloor$ après k tours (on utilise $\lfloor \lfloor n/2 \rfloor / 2 \rfloor = \lfloor n/4 \rfloor$). L'arrêt a lieu au premier k tel que $\lfloor n/2^k \rfloor \leq 1$, c'est-à-dire $n/2^k < 2$, soit $2^k > n/2$, soit $k > \log_2 n - 1$: le nombre de tours est $\lfloor \log_2 n \rfloor$. Complexité $\Theta(\log n)$.
- (d) après k tours $x = 3^k$; arrêt quand $3^k \geq n$: $k = \lceil \log_3 n \rceil$ tours, $\Theta(\log n)$ (la base du logarithme est une constante multiplicative : $\log_3 n = \log_2 n / \log_2 3$).

2. Le nombre de tours $\lceil n/c \rceil$ dépend de c , mais $1/c$ est une constante multiplicative : pour c fixé, la complexité reste $\Theta(n)$. Diviser c par deux ne change pas la classe de complexité, seulement la constante.

3. Si $t = \lfloor \log_2 n \rfloor + 1$ est le nombre de bits de n , alors $n \approx 2^t$ et :

- (c) coûte $\Theta(\log n) = \Theta(t)$: *linéaire* en la taille de l'entrée ;
- (a) coûte $\Theta(n) = \Theta(2^t)$: *exponentiel* en la taille de l'entrée.

C'est la distinction fondamentale entre complexité *pseudo-polynomiale* (polynomiale en la *valeur* de l'entrée) et complexité polynomiale (en la *taille* de l'entrée). L'algorithme (a) est en pratique inutilisable pour un entier de 200 bits.

Exercice 7 — Algorithme d'Euclide

Solution. 1. Après un tour, le couple (a, b) devient $(b, a \bmod b)$. Par définition du reste, $0 \leq$

$a \bmod b < b$. Le *variant* $V = b$ est donc un entier naturel strictement décroissant à chaque tour : la boucle termine.

2. Soit $a > b > 0$.

— Si $b \leq a/2$: $a \bmod b < b \leq a/2$.

— Si $b > a/2$: alors $a/b < 2$ et $a > b$, donc le quotient de a par b vaut exactement 1 et $a \bmod b = a - b < a - a/2 = a/2$.

Dans les deux cas $a \bmod b < a/2$.

3. Notons $(a_0, b_0) = (a, b)$ et $(a_{k+1}, b_{k+1}) = (b_k, a_k \bmod b_k)$. Alors $a_{k+2} = b_{k+1} = a_k \bmod b_k < a_k/2$ d'après 2. Donc $a_{2j} < a_0/2^j$. Comme $a_{2j} \geq 1$ tant que la boucle tourne, on a $2^j < a_0$, soit $j < \log_2 a$: le nombre de tours est $\leq 2 \log_2 a$. En remarquant qu'après le premier tour le premier argument vaut $b = \min(a, b)$ (si $a > b$), on obtient bien $O(\log \min(a, b))$.

4. Pour $k \geq 2$, $F_{k+1} = F_k + F_{k-1}$ avec $0 \leq F_{k-1} < F_k$, donc $F_{k+1} \bmod F_k = F_{k-1}$. L'appel $\text{pgcd}(F_{k+1}, F_k)$ produit donc la suite de couples

$$(F_{k+1}, F_k) \rightarrow (F_k, F_{k-1}) \rightarrow \cdots \rightarrow (F_2, F_1) = (1, 1) \rightarrow (1, 0),$$

soit exactement k tours de boucle. La borne logarithmique est donc atteinte à une constante près. Comme $F_k \sim \varphi^k / \sqrt{5}$, on a $k \sim \log_\varphi(\sqrt{5} F_k) = \frac{\ln a}{\ln \varphi} \approx 1,44 \log_2 a$: c'est le *théorème de Lamé*. Le pire des cas d'Euclide est atteint sur deux nombres de Fibonacci consécutifs.

Exercice 8 — Exponentiation rapide

Solution. 1. *Invariant* : à chaque passage au test de boucle, $r \cdot x^n = x_0^{n_0}$. *Initialisation* : $r = 1$, $x = x_0$, $n = n_0$: vrai. *Conservation* : si n est pair, $(r, x, n) \mapsto (r, x^2, n/2)$ et $r(x^2)^{n/2} = r x^n$. Si n est impair, $(r, x, n) \mapsto (rx, x^2, (n-1)/2)$ et $rx(x^2)^{(n-1)/2} = r x^n$. L'invariant est conservé. *Terminaison* : n est un entier naturel strictement décroissant (pour $n \geq 1$, $\lfloor n/2 \rfloor < n$). À la sortie $n = 0$, donc $r = r \cdot x^0 = x_0^{n_0}$.

2. Après k tours, n vaut $\lfloor n_0/2^k \rfloor$; la boucle s'arrête au premier k tel que $\lfloor n_0/2^k \rfloor = 0$, soit $2^k > n_0$: le nombre de tours est $\lfloor \log_2 n_0 \rfloor + 1$, c'est-à-dire le nombre de bits de n_0 .

3. Chaque tour effectue exactement une élévation au carré $\mathbf{x} = \mathbf{x} * \mathbf{x}$, soit $\lfloor \log_2 n \rfloor + 1$ multiplications. La multiplication $\mathbf{r} = \mathbf{r} * \mathbf{x}$ n'est effectuée que lorsque le bit courant de n vaut 1, soit $\nu(n)$ fois au total (les bits successifs de n sont exactement les valeurs de $\mathbf{n} \% 2$). D'où

$$M(n) = \lfloor \log_2 n \rfloor + 1 + \nu(n).$$

Vérification : $n = 1$ donne $0 + 1 + 1 = 2$ (une multiplication dans $\mathbf{r}$, un carré) ; $n = 3$ donne $1 + 1 + 2 = 4$. *Économie possible* : au dernier tour, l'élévation $\mathbf{x} = \mathbf{x} * \mathbf{x}$ n'est jamais réutilisée ; on peut donc la supprimer et gagner une multiplication, d'où $M'(n) = \lfloor \log_2 n \rfloor + \nu(n)$.

4. Comme $1 \leq \nu(n) \leq \lfloor \log_2 n \rfloor + 1$,

$$\log_2 n + 2 \leq M(n) \leq 2(\lfloor \log_2 n \rfloor + 1).$$

La complexité est donc $\Theta(\log n)$ dans tous les cas. Le pire cas (ν maximal) est atteint pour $n = 2^p - 1$ (écriture $11 \cdots 1$), le meilleur ($\nu(n) = 1$) pour $n = 2^p$. À comparer aux $n - 1$ multiplications de l'algorithme naïf.

Exercice 9 — Deux indices qui se croisent

Solution. 1. *Variant* : $V = d - g$. C'est un entier, positif tant que la condition $g < d$ est vraie, et chaque tour effectue soit $g \ += 1$ soit $d \ -= 1$ (ou renvoie), donc V décroît strictement de 1. La boucle termine.

2. Initialement $V = n - 1$; V décroît de 1 par tour et la boucle s'arrête dès que $V = 0$. Il y a donc au plus $n - 1$ tours. La borne est atteinte lorsque l'algorithme renvoie **None** (aucune paire de somme s), par exemple $T = [0, 1, 2, \dots, n - 1]$ et s strictement supérieur à $T[n - 2] + T[n - 1]$.

3. Chaque tour coûte $\Theta(1)$: la complexité dans le pire des cas est $\Theta(n)$. L'algorithme naïf à deux boucles **for** imbriquées teste les $\binom{n}{2}$ paires, soit $\Theta(n^2)$: le tri du tableau (supposé acquis ici) permet de passer du quadratique au linéaire.

4. Invariant : « si une solution (i, j) avec $i < j$ et $T[i] + T[j] = s$ existe, alors $g \leq i$ et $j \leq d$ ». *Initialisation* : $g = 0$ et $d = n - 1$ encadrent tout indice. *Conservation* : supposons l'invariant vrai et le tour non concluant.

— Si $T[g] + T[d] < s$: montrons $g < i$. Par l'absurde, si $g = i$, alors $T[i] + T[d] < s = T[i] + T[j]$ donne $T[d] < T[j]$, donc $d < j$ (tableau trié strictement croissant), ce qui contredit $j \leq d$. Donc $g < i$, et après $g \ += 1$ on a encore $g \leq i$; d est inchangé.

— Si $T[g] + T[d] > s$: symétriquement $d > j$, donc après $d \ -= 1$ on a encore $j \leq d$.

Conclusion : si l'algorithme renvoie **None**, c'est que $g = d$ alors qu'une solution imposerait $g \leq i < j \leq d$, donc $g < d$: contradiction. Il n'existe donc pas de solution. L'algorithme est correct.

Partie III — Le cas récursif : équation caractéristique

Exercice 10 — Tours de Hanoï et variantes

Solution. 1. $T(n) = 2T(n - 1) + 1$ pour $n \geq 1$, et $T(0) = 0$.

2. Homogène : $r = 2$, solution $A2^n$. *Particulière* : le second membre est constant et 1 n'est pas racine de $r = 2$; on cherche $T_p = c$: $c = 2c + 1 \Rightarrow c = -1$. Donc $T(n) = A2^n - 1$, et $T(0) = 0$ donne $A = 1$:

$$\boxed{T(n) = 2^n - 1}, \quad T(n) = \Theta(2^n).$$

Vérification : $T(1) = 1$, $T(2) = 3$, $T(3) = 7$. ✓

3.

(a) $T(n) = T(n - 1) + n$: par télescopage $T(n) = \sum_{k=1}^n k = \frac{n(n+1)}{2}$, soit $\Theta(n^2)$. (*Méthode caractéristique* : racine 1 simple, second membre polynomial de degré 1 ; on cherche T_p de degré $1 + 1 = 2$.)

(b) $T(n) = T(n - 1) + n^2$: $T(n) = \sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$, soit $\Theta(n^3)$.

(c) $T(n) = 2T(n-1) + n$: homogène $A2^n$; particulière $\alpha n + \beta$ donne $\alpha n + \beta = 2\alpha(n-1) + 2\beta + n$, d'où $\alpha = -1$ et $\beta = -2$. Avec $T(0) = 0$: $A = 2$ et $\boxed{T(n) = 2^{n+1} - n - 2}$, soit $\Theta(2^n)$. (*Vérification* : $T(1) = 4 - 3 = 1 = 2 \cdot 0 + 1$. ✓)

(d) $T(n) = 3T(n - 1) - 2T(n - 2)$: équation caractéristique $r^2 - 3r + 2 = 0$, racines 1 et 2. Donc $T(n) = A + B2^n$; $T(0) = 1$ et $T(1) = 3$ donnent $A = -1$, $B = 2$: $\boxed{T(n) = 2^{n+1} - 1}$, soit $\Theta(2^n)$.

Exercice 11 — Coût de Fibonacci naïf

Solution. 1. Un appel `fib(n)` avec $n \geq 2$ engendre lui-même plus les appels issus de `fib(n-1)` et `fib(n-2)` :

$$C(n) = C(n-1) + C(n-2) + 1 \quad (n \geq 2), \quad C(0) = C(1) = 1.$$

2. Homogène : $r^2 = r + 1$, racines $\varphi = \frac{1+\sqrt{5}}{2}$ et $\psi = \frac{1-\sqrt{5}}{2}$ (noter $\varphi\psi = -1$, $\varphi + \psi = 1$). Particulière : 1 n'est pas racine ($1 \neq 1 + 1$), on cherche c constante : $c = 2c + 1 \Rightarrow c = -1$. Donc

$$C(n) = A\varphi^n + B\psi^n - 1.$$

Les conditions $C(0) = C(1) = 1$ donnent $A + B = 2$ et $A\varphi + B\psi = 2$, d'où $A = \frac{2\varphi}{\sqrt{5}}$ et $B = -\frac{2\psi}{\sqrt{5}}$.

3. Ainsi

$$C(n) = \frac{2}{\sqrt{5}}(\varphi^{n+1} - \psi^{n+1}) - 1 = 2F_{n+1} - 1,$$

puisque $F_m = \frac{\varphi^m - \psi^m}{\sqrt{5}}$ (formule de Binet). Vérification : $C(0) = 2F_1 - 1 = 1$, $C(2) = 2F_3 - 1 = 3$. ✓

4. Comme $|\psi| < 1$, $\psi^{n+1} \rightarrow 0$ et $C(n) \sim \frac{2}{\sqrt{5}}\varphi^{n+1}$. La complexité de `fib` est donc $\Theta(\varphi^n)$ avec $\varphi \approx 1,618$: elle est *exponentielle*.

5. $C(80) = 2F_{81} - 1 = 75\,778\,124\,746\,287\,811 \approx 7,6 \cdot 10^{16}$ appels. À 10^9 appels par seconde : $\approx 7,6 \cdot 10^7$ secondes, soit environ **2,4 ans**. (À comparer aux 81 additions de la version itérative.)

6. Avec mémorisation, chaque valeur `fib(k)` pour $0 \leq k \leq n$ n'est calculée qu'une seule fois ; les appels suivants sont des lectures en $\Theta(1)$. Le nombre d'appels « réellement calculés » est $n + 1$ et chaque appel effectue un travail $\Theta(1)$ (hors coût des grands entiers) : la complexité devient $\Theta(n)$, pour un espace mémoire $\Theta(n)$.

Exercice 12 — Racine double et second membre

Solution.

(a) $r^2 - 4r + 4 = (r - 2)^2$: racine double $r = 2$, donc $T(n) = (A + Bn)2^n$. $T(0) = A = 1$; $T(1) = (1 + B) \cdot 2 = 6 \Rightarrow B = 2$.

$$\boxed{T(n) = (2n + 1)2^n}, \quad T(n) = \Theta(n2^n).$$

(b) $r^2 - 2r + 1 = (r - 1)^2$: racine double $r = 1$, homogène $A + Bn$. Le second membre étant constant et 1 étant racine double, on cherche $T_p = \alpha n^2$:

$$\alpha n^2 = 2\alpha(n-1)^2 - \alpha(n-2)^2 + 2 = \alpha n^2 - 2\alpha + 2 \Rightarrow \alpha = 1.$$

Donc $T(n) = A + Bn + n^2$; $T(0) = 0 \Rightarrow A = 0$, $T(1) = B + 1 = 1 \Rightarrow B = 0$:

$$\boxed{T(n) = n^2}, \quad \Theta(n^2).$$

Vérification : $T(2) = 2 \cdot 1 - 0 + 2 = 4$, $T(3) = 2 \cdot 4 - 1 + 2 = 9$. ✓

(c) C'est exactement la récurrence de l'exercice 11 : $\boxed{T(n) = 2F_{n+1} - 1}$, soit $\Theta(\varphi^n)$.

(d) $T(n) = 2T(n-1) + 2^n$: l'homogène est $A2^n$ et 2 est racine simple ; on cherche donc $T_p = \alpha n2^n$:

$$\alpha n2^n = 2\alpha(n-1)2^{n-1} + 2^n = \alpha(n-1)2^n + 2^n \Rightarrow \alpha = 1.$$

Donc $T(n) = (A + n)2^n$ et $T(0) = A = 1$:

$$\boxed{T(n) = (n + 1)2^n}, \quad \Theta(n2^n).$$

Vérification : $T(1) = 2 \cdot 1 + 2 = 4 = 2 \cdot 2^1$. ✓

Partie IV — Le cas récursif : récurrences de partition

Exercice 13 — Déroulement à la main — trois schémas de base

Solution. On pose $n = 2^k$, donc $k = \log_2 n$.

1. Recherche dichotomique. $T(2^k) = T(2^{k-1}) + 1$, $T(1) = 1$. En déroulant :

$$T(2^k) = T(2^{k-1}) + 1 = T(2^{k-2}) + 2 = \dots = T(2^0) + k = k + 1.$$

Donc $T(n) = \log_2 n + 1 = \Theta(\log n)$.

2. Somme par dichotomie. L'appel sur un segment de taille n effectue deux appels sur des segments de taille $n/2$, plus un calcul de milieu et une addition : $T(n) = 2T(n/2) + c$ avec $c > 0$ constante et $T(1) = 1$. Déroulement :

$$T(2^k) = 2T(2^{k-1}) + c = 4T(2^{k-2}) + 2c + c = \dots = 2^k T(1) + c \sum_{i=0}^{k-1} 2^i = 2^k + c(2^k - 1).$$

Donc $T(n) = (1 + c)n - c = \Theta(n)$. *Non*, cette version n'est pas meilleure que la boucle `for` naïve : même classe $\Theta(n)$, mais avec une constante plus grande (gestion de la pile d'appels) et un espace mémoire $\Theta(\log n)$ au lieu de $\Theta(1)$. Diviser pour régner n'apporte rien lorsque la recombinaison ne réduit pas le travail.

3. $T(n) = 2T(n/2) + n$, $T(1) = 1$. Avec l'arbre des appels : au niveau i ($0 \leq i \leq k$) il y a 2^i appels portant chacun sur une taille $n/2^i$, d'où un coût de niveau

$$2^i \times \frac{n}{2^i} = n \quad (\text{identique à tous les niveaux}).$$

Il y a $k = \log_2 n$ niveaux internes plus le niveau des feuilles (n feuilles de coût 1), d'où

$$T(n) = n \log_2 n + n = \Theta(n \log n).$$

4. Minimum et maximum simultanés. On découpe en deux moitiés, on obtient $(\min_1, \max_1)$ et $(\min_2, \max_2)$, puis 2 comparaisons suffisent pour recombinaison :

$$C(n) = 2C(n/2) + 2 \quad (n \geq 4), \quad C(2) = 1.$$

Posons $n = 2^k$ et $D(k) = C(2^k)$: $D(k) = 2D(k-1) + 2$, $D(1) = 1$. L'homogène est $A2^k$, la solution particulière constante vérifie $c = 2c + 2$, soit $c = -2$. Donc $D(k) = A2^k - 2$ et $D(1) = 2A - 2 = 1$ donne $A = \frac{3}{2}$:

$$D(k) = 3 \cdot 2^{k-1} - 2 \implies \boxed{C(n) = \frac{3n}{2} - 2}.$$

L'algorithme itératif naïf (un parcours pour le maximum, un pour le minimum sur les $n-1$ éléments restants) effectue $2n-3$ comparaisons. Le gain est d'environ 25 % : le nombre de comparaisons reste $\Theta(n)$, seule la constante s'améliore — ce qui est justement l'objet de ce type d'analyse fine.

Exercice 14 — Théorème général (Master Theorem)

Solution. 1.

(a) $a = 9$, $b = 3$, $\alpha = \log_3 9 = 2$. $f(n) = n = O(n^{2-\varepsilon})$ avec $\varepsilon = 1$: **cas 1**, donc $T(n) = \Theta(n^2)$.

(b) $a = 1, b = 3/2, \alpha = \log_{3/2} 1 = 0. f(n) = 1 = \Theta(n^0 \log^0 n)$: **cas 2** avec $c = 0$, donc $T(n) = \Theta(\log n)$.

(c) $a = 4, b = 2, \alpha = 2. f(n) = n^2 = \Theta(n^2 \log^0 n)$: **cas 2** avec $c = 0$, donc $T(n) = \Theta(n^2 \log n)$.

(d) $a = 3, b = 4, \alpha = \log_4 3 \approx 0,7925. f(n) = n \log n = \Omega(n^{0,7925+\varepsilon})$ pour $\varepsilon = 0,1$ par exemple. Régularité : $3f(n/4) = \frac{3}{4}n \log(n/4) \leq \frac{3}{4}n \log n = \frac{3}{4}f(n)$ avec $c = \frac{3}{4} < 1$: **cas 3**, donc $T(n) = \Theta(n \log n)$.

(e) $a = 2, b = 2, \alpha = 1, f(n) = n/\log n$. **Aucun cas ne s'applique** (voir 2). Le résultat correct est $T(n) = \Theta(n \log \log n)$: au niveau i de l'arbre, 2^i appels de taille $n/2^i$ coûtent chacun $\frac{n/2^i}{\log n - i}$, soit un coût de niveau $\frac{n}{\log n - i}$; en sommant sur $i = 0, \dots, \log n - 1$ on obtient $n H_{\log n} \sim n \ln \log n$.

2. Pour (e), $f(n) = n/\log n$ est *plus petit* que $n^\alpha = n$, donc le cas 3 est exclu ; mais il n'est pas *polynomialement* plus petit : pour tout $\varepsilon > 0$, $\frac{n/\log n}{n^{1-\varepsilon}} = \frac{n^\varepsilon}{\log n} \rightarrow +\infty$, donc $f \neq O(n^{1-\varepsilon})$ et le cas 1 est exclu. Enfin $f(n) = n \log^{-1} n$ correspondrait au cas 2 avec $c = -1 < 0$, ce qui est interdit. f tombe donc dans la « zone grise » entre les cas 1 et 2 : le théorème est muet, il faut revenir à l'arbre des appels.

3. $T(n) = 2T(n/2) + n \log n, n = 2^k$. Au niveau i : 2^i appels de taille $n/2^i$, de coût $\frac{n}{2^i} \log \frac{n}{2^i}$ chacun, soit un coût de niveau

$$2^i \cdot \frac{n}{2^i} (\log n - i) = n(k - i).$$

En sommant sur $i = 0, \dots, k - 1$:

$$T(n) = n \sum_{i=0}^{k-1} (k - i) = n \frac{k(k+1)}{2} = \Theta(n \log^2 n).$$

Remarque : ce cas relève en fait du **cas 2** avec $c = 1$ ($f(n) = \Theta(n^\alpha \log n)$ avec $\alpha = 1$), qui prédit bien $\Theta(n \log^2 n)$. L'arbre le confirme.

4. Le cas 3 affirme que le coût est dominé par la *racine* de l'arbre. Encore faut-il que le coût décroisse géométriquement quand on descend, ce qu'exprime la condition $a f(n/b) \leq c f(n)$ avec $c < 1$. Avec $f(n) = n^2 \sin^2 n$ et $a = b = 2$, f s'annule (ou devient minuscule) pour certaines valeurs de n tandis que $f(n/2)$ peut y être de l'ordre de $(n/2)^2$: l'inégalité $2f(n/2) \leq c f(n)$ est alors violée, le coût d'un niveau inférieur dépasse celui de la racine, et la conclusion $T(n) = \Theta(f(n))$ tombe en défaut. La condition de régularité est ce qui interdit ces oscillations pathologiques (elle est automatiquement vérifiée pour $f(n) = n^\beta \log^\gamma n$, cas de toutes les récurrences rencontrées en pratique).

Exercice 15 — Multiplications rapides : gagner un appel récursif

Solution. 1. Karatsuba.

(a) Avec $x = x_1 10^m + x_0$ et $y = y_1 10^m + y_0$ ($m = n/2$) :

$$xy = x_1 y_1 10^{2m} + (x_1 y_0 + x_0 y_1) 10^m + x_0 y_0.$$

L'algorithme naïf calcule les 4 produits $x_1 y_1, x_1 y_0, x_0 y_1, x_0 y_0$ de nombres à $n/2$ chiffres, puis effectue additions et décalages en $\Theta(n)$:

$$T(n) = 4T(n/2) + \Theta(n).$$

Ici $\alpha = \log_2 4 = 2$ et $f(n) = n = O(n^{2-\varepsilon})$: cas 1, donc $T(n) = \Theta(n^2)$ — on ne gagne rien sur la multiplication posée.

- (b) $(x_1 + x_0)(y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0$, d'où

$$x_1y_0 + x_0y_1 = (x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0.$$

Les trois produits $P_1 = x_1y_1$, $P_2 = x_0y_0$ et $P_3 = (x_1 + x_0)(y_1 + y_0)$ suffisent : $xy = P_1 10^{2m} + (P_3 - P_1 - P_2) 10^m + P_2$. **Trois** multiplications récursives au lieu de quatre (les additions supplémentaires restent en $\Theta(n)$).

- (c) $T(n) = 3T(n/2) + \Theta(n)$: $\alpha = \log_2 3 \approx 1,585$ et $f(n) = n = O(n^{\alpha-\epsilon})$: cas 1, donc

$$\boxed{T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1,585})}, \quad \log_2 3 = 1,585 \text{ à } 10^{-3} \text{ près.}$$

Pour $n = 10^6$ chiffres, le rapport $n^2/n^{1,585} = n^{0,415} \approx 300$: le gain est considérable.

2. Strassen.

- (a) Le produit par blocs naïf effectue 8 produits de blocs $n/2 \times n/2$ et $\Theta(n^2)$ additions : $T(n) = 8T(n/2) + \Theta(n^2)$. $\alpha = \log_2 8 = 3$, $f(n) = n^2 = O(n^{3-\epsilon})$: cas 1, donc $T(n) = \Theta(n^3)$. On retrouve bien la complexité de l'algorithme usuel à trois boucles imbriquées.
- (b) Strassen : $T(n) = 7T(n/2) + \Theta(n^2)$, $\alpha = \log_2 7 \approx 2,807$, $f(n) = n^2 = O(n^{\alpha-\epsilon})$: cas 1, donc

$$\boxed{T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2,807})}.$$

Gagner un seul produit sur huit fait passer l'exposant de 3 à 2,807.

- (c) Pour $T(n) = aT(n/2) + \Theta(n^2)$ on a $\alpha = \log_2 a$ et l'on compare α à 2 :
- $a < 4$: $\alpha < 2$, cas 3, $T(n) = \Theta(n^2)$;
 - $a = 4$: $\alpha = 2$, cas 2, $T(n) = \Theta(n^2 \log n)$;
 - $a > 4$: $\alpha > 2$, cas 1, $T(n) = \Theta(n^{\log_2 a})$.

La récurrence cesse donc d'être en $\Theta(n^2 \log n)$ dès que $a \neq 4$, et c'est à partir de $a = 5$ que la complexité dépasse strictement $\Theta(n^2 \log n)$.

Exercice 16 — Changement de variable

Solution. 1. On pose $n = 2^k$ et $S(k) = T(2^k)$. Alors $\sqrt{n} = 2^{k/2}$ et

$$S(k) = S(k/2) + 1, \quad S(1) = T(2) = 1.$$

C'est la récurrence de la recherche dichotomique : $S(k) = \log_2 k + 1$. Comme $k = \log_2 n$:

$$T(n) = \log_2 \log_2 n + 1 = \Theta(\log \log n).$$

2. Même changement de variable : $\log_2 n = k$, donc

$$S(k) = 2S(k/2) + k, \quad S(1) = 1.$$

Théorème général en la variable k : $a = 2$, $b = 2$, $\alpha = 1$, $f(k) = k = \Theta(k^1)$: cas 2 avec $c = 0$, donc $S(k) = \Theta(k \log k)$. En revenant à n :

$$\boxed{T(n) = \Theta(\log n \cdot \log \log n)}.$$

3. $T(n) = \sqrt{n}T(\sqrt{n}) + n$ devient, avec $S(k) = T(2^k)$:

$$S(k) = 2^{k/2} S(k/2) + 2^k.$$

On divise par 2^k et l'on pose $U(k) = S(k)/2^k$:

$$U(k) = \frac{2^{k/2}S(k/2)}{2^k} + 1 = \frac{S(k/2)}{2^{k/2}} + 1 = U(k/2) + 1.$$

Donc $U(k) = \Theta(\log k)$, puis $S(k) = \Theta(2^k \log k)$ et enfin

$$\boxed{T(n) = \Theta(n \log \log n)}.$$

4. La récurrence est $T(n) = T(\sqrt{n}) + \Theta(n)$. Avec $S(k) = T(2^k)$: $S(k) = S(k/2) + \Theta(2^k)$. En déroulant, le coût total est

$$\Theta\left(2^k + 2^{k/2} + 2^{k/4} + \dots\right) = \Theta\left(2^k\right),$$

car la somme est dominée par son premier terme (le second est déjà $\sqrt{2^k}$). Donc $T(n) = \Theta(n)$: le coût est entièrement concentré dans le *premier* niveau de recombinaison. C'est l'exact opposé du cas 1 du théorème général, où le coût se concentre dans les feuilles.

Fin du corrigé — 16 exercices.